

Shifter Simulator — Project Summary

Version: V2_04.06.2026.3

Owner: Decent Energy

Stack: Python 3.12 · FastAPI · PostgreSQL · MongoDB · Docker · Airflow

What This Project Does

ShifterSimulator is Decent Energy's **battery optimization and energy management platform**. It decides when home battery storage systems should charge or discharge to minimize customer energy costs and carbon footprint.

Every day, it:

1. Collects live inverter data, tariff prices, carbon intensity forecasts, and solar forecasts
2. Runs an optimization algorithm to generate a 30-minute resolution charge/discharge schedule
3. Pushes that schedule to a Shifter Service (control plane) that executes commands on physical hardware
4. Sends customers a WhatsApp summary of tomorrow's plan

It also provides a REST API for querying historical inverter telemetry.

Project Structure

```

ShifterSimulator-staging/
├── api/                                # FastAPI service (inverter telemetry API)
│   ├── main.py                        # Endpoints & API key authentication
│   ├── models.py                     # Pydantic request/response schemas
│   └── database.py                   # PostgreSQL queries
├── app/
│   ├── shifter/                      # Battery optimization engine (core)
│   │   ├── shifter_orchestrator.py
│   │   ├── shifter_context.py
│   │   ├── schedule_validator.py
│   │   ├── accounts_api_client.py
│   │   └── models/                  # Domain models (installation, battery,
inverter, etc.)
│   └── optimisers/                  # DP scheduler, simple scheduler, day
scheduler
│   └── pipelines/                   # forecast.py, historical.py
│       ├── data_retrieval/         # External API integrations (Octopus,
inverters, carbon, solar)
│       ├── db_functions/           # PostgreSQL & MongoDB helpers
│       ├── crm/                   # Customer sync, HubSpot, tariff validation
│       └── general_functions/      # Logging, messaging (WhatsApp, Slack),
battery utilities
├── daily_data_processors/           # Airflow entry-point scripts (run daily)
├── monitoring_scripts/              # Health checks and accuracy reports
├── notebooks/                      # Jupyter notebooks for analysis
└── json_to_db_migration.py          # Seeds lookup tables into PostgreSQL

```

└─ run_api.py	# FastAPI launcher
└─ Dockerfile	
└─ requirements.txt	
└─ .github/workflows/	# CI/CD pipelines

Technology Stack

Category	Technology
Language	Python 3.12
API Framework	FastAPI 0.104 + Uvicorn 0.24
Data Processing	Pandas 2.3, NumPy 2.3
Data Validation	Pydantic 2.11
Primary Database	PostgreSQL (SQLAlchemy 2.0)
Secondary Database	MongoDB
Messaging	Twilio (WhatsApp), Slack
Error Tracking	Sentry SDK 2.20
Scheduling	Apache Airflow (external)
Containers	Docker
CI/CD	GitHub Actions
Geospatial	GeoPandas, Shapely
Daylight	Astral 3.2

Core Components

1. Battery Optimization Engine (app/shifter/)

The heart of the system. Takes a customer's installation configuration and time-series data and produces an optimal battery schedule.

Key classes:

Class	File	Purpose
ShifterOrchestrator	shifter_orchestrator.py	Coordinates scheduling across one or more days; routes to the right optimizer
ShifterContext	shifter_context.py	Bundles installation config + time-series DataFrame + live SOC for one optimization run
InstallationContext	models/installation.py	Complete customer installation config (inverter, battery, tariffs, location, shifter level)

Class	File	Purpose
DynamicProgrammingScheduler	optimisers/ dp_scheduler.py	Optimal scheduling via backward/forward DP pass
SimpleScheduler	optimisers/ simple_scheduler.py	Rule-based heuristic fallback (charge cheap, discharge expensive)
DayScheduler	optimisers/ day_scheduler.py	Single-day wrapper with 3-attempt retry logic

Scheduling decision logic:

```
ShifterOrchestrator.run()
├─ shifter_level == "simple_shifter"?
│   └─ YES → SimpleScheduler (rule-based)
│       └─ NO → DynamicProgrammingScheduler
│           └─ Try multi-day DP
│               └─ Fall back to per-day DP → then SimpleScheduler if all else
fails
```

Optimization objective (DP):

$\text{combined_cost} = \alpha \times \text{price_per_kWh} + (1 - \alpha) \times \text{carbon_intensity_gCO2}$

- $\alpha = 1.0$ → pure price minimization
- $\alpha = 0.0$ → pure carbon minimization
- $\alpha = 0.8$ → default (80% price, 20% carbon)

Battery actions:

- Charge battery — draw from grid to charge battery
- Use battery — discharge battery to cover load
- Defer use — wait for better price slot
- Allow solar — let excess PV generation charge battery
- Do nothing — no action (stub/padding slots)

Battery charging constraints:

1. Hardware cap (inverter's rated AC power)
2. CC/CV curve — empirical fitted model per customer: constant current below knee_soc, exponential taper above it
3. Peak reserve — battery must hold enough charge during evening peak (4–9 PM) to cover remaining peak demand

2. Inverter Telemetry API (api/)

A small FastAPI service (port 3003) for querying historical inverter data.

Endpoint	Method	Auth	Description
/	GET	None	Health check
/inverter/reporting-data	GET	X-Api-Key header	Returns half-hourly inverter telemetry
/_sentry-test	GET	None	Triggers a test Sentry error

Query parameters for /inverter/reporting-data:

- `ci` (int) — Customer ID
- `start_ts` (datetime ISO) — Start of window
- `end_ts` (datetime ISO) — End of window

Response fields per slot: `customer_id`, `timestamp`, `consumption_kwh`, `pv_kwh`, `battery_diff_kwh`, `grid_import_kwh`, `grid_export_kwh`

Data source: PostgreSQL view `inverter_data.inverter_reporting_data`

3. Data Retrieval Layer (app/data_retrieval/)

Integrates with multiple external APIs to collect input data for the optimizer.

Module	Source	Data Collected
<code>octopus/octopus_pricing.py</code>	Octopus Energy API	Half-hourly import/export tariff rates
<code>octopus/octopus_account_info.py</code>	Octopus Energy API	Customer tariff history
<code>octopus/octopus_agile_available.py</code>	Octopus Energy API	Whether Agile tariff is active
<code>carbon_intensity_data_retrieval.py</code>	<code>carbonintensity.org.uk</code>	14-day regional carbon intensity forecast
<code>retrieve_solar_forecasts.py</code>	PostgreSQL (Solcast-backed)	Calibrated PV generation forecast
<code>retrieve_inverter_data_any.py</code>	Solis / FoxESS / GivEnergy / SolarEdge / Eleven	Live SOC, power readings
<code>pricing_data_any.py</code>	Octopus + PostgreSQL	Combined import/export pricing for DP input

Carbon data handling: Fills missing values via linear interpolation (up to 4 consecutive 30-min slots). If a regional code is missing, falls back to national average.

4. Daily Data Processors (daily_data_processors/)

Airflow-invoked scripts that run on a schedule. Each is a standalone entry point.

Script	When run	What it does
forecasting_shifter.py	Daily (morning)	Runs the full forecast pipeline for all active customers
inverter_daily_data_processor.py	Daily	Collects inverter telemetry from cloud APIs, stores to PostgreSQL
pricing_daily_data_processor.py	Daily	Fetches tariff rates, stores to PostgreSQL
carbon_daily_data_processor.py	Daily	Fetches carbon intensity, stores to PostgreSQL
meter_daily_data_processor.py	Daily	Ingests consumption from smart meters
historical_shifter.py	On-demand	Backtests optimizer against historical meter data
new_customer_check.py	Daily	Validates onboarding data integrity, syncs tariffs
battery_sizing_processor.py	On-demand	Analyzes battery utilization, recommends sizing changes
peak_failsafe.py	Daily	Enforces demand-side management (peak export caps)

5. Forecast Pipeline (app/shifter/pipelines/forecast.py)

The top-level daily orchestrator:

```
ForecastPipeline.run()
1. Fetch all active installations from Accounts API
2. For each installation:
  a. Get live inverter SOC and power from cloud API
  b. Retrieve consumption forecast (from predictions)
  c. Retrieve pricing data (Octopus or PostgreSQL cache)
  d. Retrieve carbon intensity (PostgreSQL)
  e. Retrieve PV forecast (PostgreSQL)
  f. Build InstallationContext from Accounts API
  g. Create ShifterContext(installation, data_df, live_soc)
  h. Run ShifterOrchestrator → get schedule
  i. Validate schedule (SOC bounds, rate limits, action alignment)
  j. Push schedule to Shifter Service (MongoDB)
  k. Send WhatsApp notification to customer
3. Log errors and finalize
```

6. CRM & Account Integration (app/crm/)

Module	Purpose
crm_customer_comparison.py	Compares Octopus tariff records vs internal Accounts API, syncs mismatches
check_variable_information.py	Validates customer data completeness before scheduling
dno_region_calculation.py	Maps DNO area (e.g., "SECR") to carbon intensity region
process_location_info.py	Geocodes postcodes, stores latitude/longitude
hubspot_send_data.py	Syncs customer events to HubSpot CRM

7. Notifications & Messaging (app/general_functions/messaging_functions/)

WhatsApp (Twilio):

- `send_whatsapp_actions_message()` — Sends daily schedule summary: charge/discharge cycle count, average cost per kWh, and timing of the first 3 actions
- `send_whatsapp_ev_message()` — EV low-price charging alerts
- Template-based using Twilio content SIDs (6 variables per message)
- Only sends in production (`PC_NAME != "decent_staging"`)

Slack:

- Operational alerts for ERROR/WARNING events
- New customer signup notifications
- Tariff mismatch alerts

Shifter Service (`send_to_shifter_service.py`):

- Posts schedule instructions to control-plane MongoDB
- Reformats action enums to control-plane schema
- Handles bulk insert with per-record failure reporting

8. Monitoring Scripts (monitoring_scripts/)

Script	Checks
battery_soc_deviation.py	Actual vs predicted SOC divergence
carbon_data_checker.py	Carbon intensity data completeness
check_for_high_prices.py	Unusual pricing alerts

Script	Checks
check_for_low_prices.py	Missed arbitrage opportunities
daily_forecast_accuracy_report.py	Predicted vs actual savings comparison
new_customer_signups.py	Onboarding pipeline tracking
savings_comparison_report.py	Financial analysis reports

Domain Data Models

InstallationContext (the main config object)

Field	Type	Description
customer_id	int	Unique customer identifier
shifter_level	str	"simple_shifter" or "smart_shifter"
cost_bias (alpha)	float	0.0–1.0, weight of price vs carbon
inverter	Inverter	AC capacity (kW), charge/discharge curves, serial
battery	BatteryDevice	Capacity (kWh), min SOC %, capacity history
import_connection	GridConnection	Import MPAN, supplier, tariff history
export_connection	GridConnection	Export MPAN, export tariff history
solar_array	SolarArray	PV capacity (kWp), pitch, azimuth
customer	Customer	Name, email, WhatsApp number
integration	InverterIntegration	Cloud API credentials (Solis, FoxESS, etc.)

Time-Series DataFrame Schema (input to optimizer)

Each row = one 30-minute slot in UTC:

Column	Unit	Description
Timestamp	UTC datetime	Slot start time
Consumption	kWh	Expected grid import
Price	p/kWh	Import tariff rate

Column	Unit	Description
Cost	£	= Consumption × Price
Carbon_intensity	gCO2/kWh	Regional carbon forecast
PV_forecast	kWh	Calibrated solar generation forecast
Export_Price	p/kWh	Export tariff rate (optional)
Product	str	Tariff code/name

PostgreSQL Tables

Table	Purpose
inverter_data.inverter_reporting_data	Half-hourly telemetry view
data_sources.tariff_info	Tariff code registry
data_sources.carbon_intensity	Carbon forecast by region and datetime
data_sources.decent_solar_forecast	Calibrated PV forecast by customer
data_sources.customer_charge_curve	Fitted CC/CV battery charge models

Environment Configuration

Set via .env file. Key variables:

Variable	Purpose
API_TOKEN	FastAPI X-Api-Key header value
PGSQL_HOST/PORT/DB/USER/PASS	PostgreSQL connection
MONGODB_ACCOUNTS_URI	MongoDB for customer metadata
MONGODB_STATS_URI	MongoDB for instruction statistics
ACCOUNTS_API_URL/TOKEN	Internal Accounts API
SHIFTERSERVICE_URL/API_KEY	Internal control-plane (Shifter Service)
PC_NAME	Deployment identifier (decent_production enables live messaging)
TWILIO_SID/TOKEN/NUMBER	WhatsApp credentials
TWILIO_ACTIONS_TEMPLATE	Twilio content template SID
SLACK_TOKEN	Slack bot token for alerts
SENTRY_DSN	Error tracking endpoint

Variable	Purpose
SOLCAST_API_KEY	Solar irradiance API
DICT_FILES	Path to shared JSON lookup dictionaries

How to Run

```
# Start the inverter telemetry API (port 3003)
python run_api.py

# Run the daily forecast pipeline
python daily_data_processors/forecasting_shifter.py

# Collect inverter data for a specific date
python daily_data_processors/inverter_daily_data_processor.py 2026-06-09

# Collect pricing data
python daily_data_processors/pricing_daily_data_processor.py 2026-06-09

# Run historical backtest
python daily_data_processors/historical_shifter.py 2026-06-09

# Seed lookup tables into PostgreSQL
python json_to_db_migration.py

# Docker build
docker build -t shifter-simulator .
```

CI/CD Pipelines (.github/workflows/)

Workflow	Trigger	Action
deploy-staging.yaml	Manual	SSH deploy to Linode staging VPS
deploy-production.yaml	Manual	SSH deploy to production VPS
deploy-development.yaml	Manual	SSH deploy to development VPS
deploy-to-registry.yaml	Manual	Push Docker image to container registry
bump-version-on-pr.yaml	PR open	Auto-increment version string
pylint.yaml	PR / push	Python linting checks
dependabot.yml	Scheduled	Automated dependency updates

Supported Inverter Brands

Brand	Integration Method
Solis	REST API (serial + API key)
FoxESS	foxess-cloud Python library
GivEnergy	Cloud REST API
SolarEdge	REST API
Eleven	REST API

Observability

- **Sentry:** Captures unhandled exceptions and performance traces (10% sample rate). Initialized per component (api, pipelines, processors).
- **Logging:** Airflow task logger when running inside Airflow; stdout/stderr when standalone. Slack alerts auto-sent for ERROR/WARNING in production.
- **Schedule Validation:** After every optimization run, checks SOC bounds, direction alignment, rate limits, and NaN actions. All checks are warning-only (a schedule is never rejected, only flagged).

External APIs Consumed

API	Purpose
Octopus Energy	Tariff history, meter readings, Agile pricing
carbonintensity.org.uk	Regional UK carbon intensity forecast
Solis / FoxESS / GivEnergy / SolarEdge / Eleven	Inverter live data
Solcast	Solar irradiance forecast
Accounts API (internal)	Customer & installation config
Shifter Service (internal)	Control-plane schedule storage
Twilio	WhatsApp messaging
Slack	Operational alerts
HubSpot	CRM sync
Jira	Ticket automation
SharePoint	Document uploads

Known Limitations & TODOs (noted in code)

- Charge curves are keyed per **customer**, not per **device** — a customer with 2 inverters gets the same curve applied to both (noted as a TODO in `models/inverter.py`)
- Consumption forecasts are assumed to be provided by an external caller; `ShifterSimulator` does not generate them internally
- Schedule validation checks are all warning-only and never block a schedule from being pushed